

Home Assignment 1: LangChain Memory

Use the `qa-bot-ts_langchain` application and its Postman collection to practise and understand how LangChain memory works. Test the application with multiple questions and verify whether it remembers the previous conversation context.

Home Assignment 1: LangChain Memory

Objective

Use the `qa-bot-ts_langchain` application and its Postman collection to practise and understand how **LangChain memory** works. Test the application with multiple questions across a single conversation and verify whether it remembers the previous conversation context.

Application Overview

`qa-bot-ts_langchain` is a TypeScript RAG (Retrieval-Augmented Generation) API for searching resumes. It exposes a conversational chat endpoint that maintains per-conversation memory so follow-up questions can refer back to earlier turns.

- **Runtime:** Node.js + Express (TypeScript)
- **LLM Provider / Model:** Groq - `openai/gpt-oss-120b`
- **Base URL:** `http://localhost:8787`
- **Start command:** `npm run dev`

How LangChain Memory Works in This App

Memory is implemented in `src/lib/memory/chatMemory.ts` using LangChain's **`BufferMemory`**, wrapped by a `ChatMemoryManager` and tracked in an in-memory `ConversationStore`.

Key characteristics:

Aspect	Behaviour
--------	-----------

```

|-----|-----|
| Memory type | LangChain `BufferMemory` with `ChatMessageHistory` |
| Keyed by | `conversationId` (each conversation has its own memory
manager) |
| Message limit | Last **10 messages** (~5 exchanges); older messages are
trimmed |
| Persistence | **In-memory (RAM) only** – restarting the server clears
all conversations |
| Continuity rule | Reusing the same `conversationId` continues a
conversation; omitting it starts a new one |

**Endpoints used:**

- `POST /chat` – conversational query. Body: `{ message, conversationId?,
topK?, includeHistory? }`. If `conversationId` is omitted, a new
conversation is created and its ID is returned.
- `POST /chat/history` – returns the stored message history for a
`conversationId`.
- `DELETE /chat/{conversationId}` – deletes a conversation and clears its
memory.

**How memory is proven:** When the same `conversationId` is reused, the
app recognises refinement queries (e.g. "From those...", "Now show me
only...") and applies a **conversational filter on the cached results from
the previous turn** (`searchType: "filter"`), and the `messageCount` grows
with each turn. This is only possible because the earlier context was
remembered.

---

## Test Setup

- **Postman collection:** `QA Bot TS - LangChain API Copy` → folder
`Conversational Chat (Memory)`
- **Requests exercised:** Start New Conversation, Continue Conversation
(Follow-up), Filter Results (Conversational), Get Conversation History,
Delete Conversation
- **Conversation ID used throughout:** `conv_1789743437969_6xn5trl0j`

---

```

Phase 1 — Start New Conversation

****Request**** — `POST /chat`

```json

```
{
 "message": "Find Selenium testers with more than 5 years of experience",
 "topK": 7,
 "includeHistory": false
}
```

```

****Response**** — `200 OK`

```json

```
{
 "response": "**Selenium-tester candidates with > 5 years of experience**
... (summary: none of the listed profiles explicitly state Selenium + >5
years; suggests opening full resumes or refining the query)",
 "conversationId": "conv_1789743437969_6xn5trl0j",
 "messageCount": 2,
 "model": "openai/gpt-oss-120b",
 "provider": "groq",
 "searchMetadata": {
 "query": "Find Selenium testers with more than 5 years of experience",
 "searchType": "hybrid",
 "resultCount": 7,
 "duration": 9605
 }
}
```

```

****Observation:**** A new conversation was created and a `conversationId` (`conv\_1789743437969\_6xn5trl0j`) was returned. `messageCount` is 2 (1 user + 1 AI). `searchType` is `hybrid` (a fresh full search). This ID is reused in all subsequent phases to test memory continuity.

Phase 2 – Continue Conversation (Follow-up)

****Request**** – `POST /chat`

```
```json
{
 "message": "From those, filter only candidates only above 10 years",
 "conversationId": "conv_1789743437969_6xn5trl0j"
}
```
```

****Response**** – `200 OK`

```
```json
{
 "response": "Based on your filter (**candidates with above 10 years of experience**), none of the 7 candidates provide sufficient experience information to meet the requirement. All candidates lack explicit years of experience in their profiles, so they are marked as non-matching.",
 "conversationId": "conv_1789743437969_6xn5trl0j",
 "messageCount": 4,
 "model": "openai/gpt-oss-120b",
 "provider": "groq",
 "searchResults": [],
 "searchMetadata": {
 "query": "From those, filter only candidates only above 10 years",
 "searchType": "filter",
 "resultCount": 0,
 "duration": 1760
 }
}
```
```

****Observation (memory proven):****

- Same `conversationId`; `messageCount` grew from 2 → ****4****, so the turn was appended to the existing conversation instead of starting fresh.
- `searchType` is ****`filter`**** (not `hybrid`), meaning the app recognised "From those..." as a refinement and filtered the cached results from Phase 1.

- The AI response refers to "the **7 candidates**" - i.e. the exact result set produced in Phase 1. This confirms the previous context was remembered.

Phase 3 - Filter Results (Conversational)

****Request**** - `POST /chat`

```json

```
{
 "message": "Now show me only the ones in Chennai",
 "conversationId": "conv_1789743437969_6xn5trl0j"
}
```

```

****Response**** - `200 OK`

```json

```
{
 "response": "Based on your request (candidates located in Chennai), none of the 7 candidates have a confirmed Chennai location in their profiles or resume excerpts. All candidates lack location information, so we cannot verify they are based in Chennai.",
 "conversationId": "conv_1789743437969_6xn5trl0j",
 "messageCount": 6,
 "model": "openai/gpt-oss-120b",
 "provider": "groq",
 "searchResults": [],
 "searchMetadata": {
 "query": "Now show me only the ones in Chennai",
 "searchType": "filter",
 "resultCount": 0,
 "duration": 2077
 }
}
```

```

****Observation (memory proven):****

```
- `messageCount` climbed to **6** (2 → 4 → 6), confirming all turns are retained in the same conversation buffer.  
- `searchType` is again **`filter`** – still refining the remembered result set rather than launching a new search.  
- Three turns in, the AI still refers to "the **7 candidates**" from Phase 1, demonstrating sustained context retention.
```

Phase 4 – Get Conversation History

```
**Request** – `POST /chat/history`
```

```
```json
```

```
{
 "conversationId": "conv_1789743437969_6xn5trl0j"
}
```

```
```
```

```
**Response** – `200 OK`
```

```
```json
```

```
{
 "conversationId": "conv_1789743437969_6xn5trl0j",
 "messages": [
 { "role": "user", "content": "Find Selenium testers with more than 5 years of experience" },
 { "role": "assistant", "content": "**Selenium-tester candidates with > 5 years of experience** ... (full summary)" },
 { "role": "user", "content": "From those, filter only candidates only above 10 years" },
 { "role": "assistant", "content": "Based on your filter (**candidates with above 10 years of experience**), none of the 7 candidates ..." },
 { "role": "user", "content": "Now show me only the ones in Chennai" },
 { "role": "assistant", "content": "Based on your request (**candidates located in Chennai**), none of the 7 candidates ..." }
],
 "messageCount": 6
}
```

**\*\*Observation (definitive proof):\*\*** The history endpoint returned **\*\*all 6 messages\*\*** – 3 user turns and 3 AI responses – in the exact order they occurred. This confirms the ``BufferMemory`` retained the complete conversation and that it is retrievable by ``conversationId``.

---

## ## Phase 5 – Delete Conversation

**\*\*Request\*\*** – ``DELETE /chat/conv_1789743437969_6xn5trl0j``

**\*\*Response\*\*** – ``200 OK`` (11 ms)

```
```json
{
  "success": true,
  "conversationId": "conv_1789743437969_6xn5trl0j",
  "message": "Conversation deleted successfully"
}
```
```

**\*\*Observation:\*\*** The conversation and its memory were cleared. A subsequent ``Get Conversation History`` request for this ID would now return ``404 Not Found``, proving that memory was tied to the ``conversationId`` and can be explicitly cleared.

---

## ## Results Summary

| Phase | Request                | conversationId                         | messageCount | searchType | Outcome                                    |
|-------|------------------------|----------------------------------------|--------------|------------|--------------------------------------------|
| ----- | -----                  | -----                                  | -----        | -----      | -----                                      |
| 1     | Start New Conversation | <code>`conv_...6xn5trl0j`</code> (new) | 2            | hybrid     | New conversation + ID created              |
| 2     | Continue (Follow-up)   | same                                   | 4            | filter     | Understood "From those" → filtered prior 7 |
| 3     | Filter Results         | same                                   | 6            | filter     | Still referencing the same 7 candidates    |
| 4     | Get History            | same                                   | 6            | –          | Returned all 6 messages in order           |

```
| 5 | Delete Conversation | same | - | - | Memory cleared successfully |
```

```

```

## ## Negative Validation

Negative validation confirms the app has **no memory** when a valid ``conversationId`` is not supplied or no longer exists – the mirror image of the positive tests above.

### ### Negative Test – Get History on a deleted / non-existent conversationId

**Request** – ``POST /chat/history``

````json`

```
{
  "conversationId": "conv_1789743437969_6xn5trl0j"
}
```

`````

**Response** – ``404 Not Found`` (7 ms)

````json`

```
{
  "error": "Conversation not found: conv_1789743437969_6xn5trl0j"
}
```

`````

**Observation:** This is the same ``conversationId`` that returned 6 messages in Phase 4. After it was deleted in Phase 5, the history endpoint now responds with ``404 Not Found``. This proves that:

1. Memory is **tied to a valid `conversationId`** – an unknown or cleared ID has no retrievable context.
2. **Deleting a conversation truly clears its memory** (before/after contrast: Phase 4 returned 6 messages → after delete the same ID 404s).

### ### Additional negative checks (optional)

```
| Test | Request | Expected result | Proves |
```



```
|-----|-----|-----|-----|
| Follow-up with **no** `conversationId` | `POST /chat` with body `{
"message": "From those, filter only candidates only above 10 years" }` |
New `conversationId`, `messageCount: 2`, `searchType: "hybrid"` (not
`filter`) | Without an ID there is no remembered context; "those" cannot
be resolved and a fresh search runs |
| History with a **fake** ID | `POST /chat/history` with `{
"conversationId": "conv_does_not_exist_123" }` | `404 Not Found` | Memory
requires a real, existing conversation |
```

---

## ## Conclusion

The application **\*\*successfully demonstrates LangChain memory\*\***:

1. **\*\*Context retention\*\*** – Follow-up queries referencing "those" and "the ones" were correctly interpreted against the results of the first query, three turns deep.
2. **\*\*Conversation continuity\*\*** – `messageCount` increased consistently (2 → 4 → 6) within a single `conversationId`, and the history endpoint returned the full ordered transcript.
3. **\*\*Refinement behaviour\*\*** – After the initial `hybrid` search, follow-ups switched to `searchType: "filter"`, applying the conversational filter over the remembered/cached results rather than re-running a fresh search.
4. **\*\*Memory scoping\*\*** – Memory is keyed by `conversationId` and stored in RAM; it can be explicitly cleared via the delete endpoint (confirmed in Phase 5).

**\*\*Note:\*\*** All searches returned `resultCount: 0` for the experience/location filters because the ingested resume records lacked populated structured fields (experience, location). This is a **\*\*data/ingestion limitation, not a memory limitation\*\*** – the memory behaviour (context retention across turns) was still clearly and correctly demonstrated throughout.